Gradient Checking

A Beginner's Companion Guide

How to prove your backpropagation is correct — the components, the formula, and why it works

About This Guide

Proving That Backpropagation Actually Works

A plain-language walkthrough of gradient checking — and the questions it answers

Backpropagation is the engine that trains every neural network, and it is also notoriously easy to get subtly wrong. A single misplaced factor or a wrong index can leave your gradients *almost* right — close enough that the network still seems to train, but wrong enough that it never works as well as it should. Worse, these bugs are silent: nothing crashes, no error appears, the cost still goes down. So how can you ever be *sure* your backpropagation is correct?

That is exactly the question this assignment answers, and it does so with a memorable scenario. You have been hired to build a fraud-detection model for a mobile-payments company. Because the stakes are high, the CEO demands proof: "Give me proof that your backpropagation is actually working!" The tool you reach for is **gradient checking** — a clever, almost embarrassingly simple technique that verifies your hand-derived gradients against a second, independent estimate. This guide walks through the code you wrote, defines each component in plain terms, and answers the questions the assignment raises along the way.

> **HOW TO READ THIS GUIDE**
>
> Each section follows the same rhythm: the concept in everyday language, the code that implements it, and a short note on where the idea fits in the wider world of deep learning. You need no mathematics beyond multiplication, subtraction, and the idea of a slope. The questions posed in the notes — what gradient checking means, how it proves backprop is working, and how you actually use it — are answered directly in the sections where they arise, and gathered again at the end.

1. What Gradient Checking Is, and the Big Idea Behind It

Gradient checking rests on one beautifully simple observation. Your network has two separate pieces of code that touch the gradient. **Forward propagation** computes the cost `J` — and forward propagation is easy to write, so you are confident it is correct. **Backward propagation** computes the gradient of that cost — and backprop is hard to write, so you are *not* confident. Gradient checking uses the code you trust (forward propagation) to test the code you do not (backward propagation).

> **KEY CONCEPT: GRADIENT CHECKING**
>
> Gradient checking is a debugging technique that verifies your backpropagation code by comparing the gradient it produces against a second, independent estimate of the same gradient computed only from the cost function. If the two agree to many decimal places, your backprop is almost certainly correct. If they disagree, there is a bug. It is a test, not a part of training — a way to earn confidence in code you will then trust.

The key insight is that there are two completely different ways to find a gradient, and they should agree. The first is **backpropagation** — the fast, calculus-based method you actually use to train. The second is a **numerical approximation** — a slow but dead-simple method that estimates the slope directly from the definition of a

derivative, using only the cost function. Because the two methods share no code, a bug in backprop will not appear in the approximation, so any disagreement between them exposes the bug.

2. The Numerical Approximation: A Derivative Is Just a Slope

To understand the approximation, recall what a derivative *is*. It is the slope of a function — how much the output changes when you nudge the input a tiny bit. The formal definition is:

```
    dJ/dtheta  =  lim   ( J(theta + epsilon) - J(theta - epsilon) ) / (2 * epsilon)
                 epsilon->0
```

In words: nudge the parameter up by a tiny amount `epsilon`, nudge it down by the same amount, see how much the cost `J` changed, and divide by the total distance you moved. That ratio *is* the slope. The "limit as epsilon goes to zero" simply means "when epsilon is really, really small." We cannot make epsilon truly zero on a computer, but a tiny value like `1e-7` gets us extremely close.

> **KEY CONCEPT: THE TWO-SIDED DIFFERENCE**
>
> Notice the formula nudges *both* directions — up to `theta + epsilon` and down to `theta - epsilon` — and divides by `2 * epsilon`. This is the **two-sided** (or centered) difference. You could instead nudge only one way, but the two-sided version is far more accurate: its error shrinks with the *square* of epsilon rather than just epsilon. That extra accuracy is exactly what you need when checking a gradient to seven decimal places, so gradient checking always uses the two-sided form.

The crucial point is that this approximation uses *only* the cost function `J` — which comes from forward propagation, the code you trust. It never touches your backprop code. That independence is what makes it a valid test.

3. The Components, Seen in One Dimension

The assignment builds gradient checking on the simplest possible model: a one-parameter function `J(theta) = theta * x`. Tiny as it is, it contains every component of the full technique. There are three pieces.

The first component is **forward propagation** — computing the cost. For this model it is a single multiplication:

```
def forward_propagation(x, theta):
    # J(theta) = theta * x  -- this is the cost we trust
    J = theta * x
    return J
```

The second component is **backward propagation** — the gradient you want to verify. The derivative of `theta * x` with respect to `theta` is simply `x`:

```
def backward_propagation(x, theta):
    # the gradient dJ/dtheta. For J = theta * x, this is just x.
```

```
    dtheta = x
    return dtheta
```

The third and central component is **gradient_check** itself, which ties the other two together. It builds the numerical approximation in five small steps, calls backprop for the "real" gradient, and then measures how far apart the two are:

```
def gradient_check(x, theta, epsilon=1e-7, print_msg=False):
    # --- Build the numerical approximation using only forward_propagation ---
    theta_plus  = theta + epsilon                # Step 1: nudge up
    theta_minus = theta - epsilon                # Step 2: nudge down
    J_plus  = forward_propagation(x, theta_plus)  # Step 3: cost after nudging up
    J_minus = forward_propagation(x, theta_minus) # Step 4: cost after nudging down
    gradapprox = (J_plus - J_minus) / (2 * epsilon)   # Step 5: the slope

    # --- The gradient we want to check, from backprop ---
    grad = backward_propagation(x, theta)

    # --- Relative difference between the two estimates ---
    numerator   = abs(grad - gradapprox)          # how far apart are they?
    denominator = abs(grad) + abs(gradapprox)     # scale, so the result is a ratio
    difference  = numerator / denominator

    if print_msg:
        if difference > 2e-7:
            print("Mistake in backward propagation! difference = " + str(difference))
        else:
            print("Backward propagation works perfectly! difference = " + str(difference))
    return difference
```

Run with `x = 3, theta = 4`, this reports a difference of about `7.8e-11` — far below the `1e-7` threshold — and prints "Your backward propagation works perfectly fine!" The numerical slope and the backprop gradient agree to ten decimal places, so you can trust the gradient.

> **THE THREE COMPONENTS IN A NUTSHELL**
>
> Every gradient check, however large, is built from these same three pieces: **forward propagation** to compute the cost `J` (trusted), **backward propagation** to compute the gradient (under test), and the **check** itself, which approximates the gradient numerically from `J` alone and compares it to backprop's answer. The approximation never uses the backprop code — that independence is the whole point.

4. Why a Relative Difference, and What the Threshold Means

The check does not ask "are the two gradients exactly equal?" — they never will be, because the approximation is only approximate. Instead it computes a **relative difference**: the distance between the two estimates, divided by their combined size. Dividing by the size turns the answer into a ratio, so it is meaningful whether the gradients happen to be tiny or huge.

> **KEY CONCEPT: READING THE DIFFERENCE**
>
> The relative difference is a single number that grades the agreement. A value around `1e-7` or smaller means the gradient is almost certainly correct. Around `1e-5` is a yellow flag — probably fine, but worth a careful look. Anything around `1e-3` or larger is a red flag: there is very likely a bug. Smaller is better, and the assignment uses a pass threshold of `2e-7`.

This answers a natural question — *why compute a difference at all, rather than just printing the two gradients?* Because a single graded number gives a clear, automatic verdict: pass or fail. You do not have to eyeball long lists of numbers; the difference collapses the entire comparison into one value you can test against a threshold.

5. Scaling Up: Checking a Whole Network

A real network does not have one parameter — it has many weight matrices `W` and bias vectors `b` spread across its layers. The idea is identical, but now you must check *every* parameter. Two helper steps make this manageable.

> **KEY CONCEPT: FLATTENING PARAMETERS INTO A VECTOR**
>
> To check a whole network, you first reshape every weight matrix and bias vector into one long column — a giant vector called `theta` — by unrolling each and stacking them. The helper `dictionary_to_vector()` does this; its inverse `vector_to_dictionary()` rebuilds the original shapes. Backprop's gradients are flattened the same way into `grad`. Now both the parameters and their gradients are simple lists of numbers, lined up in the same order, ready to compare one entry at a time.

With everything flattened, `gradient_check_n` walks through the parameter vector one entry at a time. For each parameter it nudges *only that one* up by epsilon (leaving all others fixed), records the cost, nudges it down, records the cost again, and forms the same two-sided slope as before. The result is a whole vector `gradapprox` of numerical gradients, one per parameter:

```
for i in range(num_parameters):
    # --- J_plus[i]: nudge ONLY parameter i upward ---
    theta_plus = np.copy(parameters_values)
    theta_plus[i] = theta_plus[i] + epsilon
    J_plus[i], _ = forward_propagation_n(X, Y, vector_to_dictionary(theta_plus))

    # --- J_minus[i]: nudge ONLY parameter i downward ---
    theta_minus = np.copy(parameters_values)
    theta_minus[i] = theta_minus[i] - epsilon
    J_minus[i], _ = forward_propagation_n(X, Y, vector_to_dictionary(theta_minus))

    # --- the two-sided slope for parameter i ---
    gradapprox[i] = (J_plus[i] - J_minus[i]) / (2 * epsilon)

# --- compare the whole approximation vector to backprop's grad vector ---
numerator   = np.linalg.norm(grad - gradapprox)              # Euclidean distance
```

```
denominator = np.linalg.norm(grad) + np.linalg.norm(gradapprox)
difference  = numerator / denominator
```

The only change from the 1D case is at the end: because we are now comparing two *vectors* rather than two numbers, the distance between them is measured with `np.linalg.norm` — the Euclidean length of their difference — in place of the simple absolute value. The logic is otherwise exactly the same.

6. How It Proves Backprop Is Working — By Catching a Real Bug

This is where the assignment makes the lesson unforgettable, and it directly answers the notes' question: *how do you show backpropagation is working, and what does that prove?* The provided `backward_propagation_n` contains two deliberate bugs. Run gradient checking on it and you get:

> **THE CHECK FIRES**
>
> **There is a mistake in the backward propagation! difference = 0.285**

A difference of `0.285` is enormous — nowhere near the `2e-7` threshold — so the check loudly declares the gradient wrong. This is gradient checking doing its job: the numerical estimate (built only from the trusted cost) disagrees sharply with backprop, exposing bugs that ran silently without any crash. The buggy lines were:

```
    dW2 = 1. / m * np.dot(dZ2, A1.T) * 2        # BUG: stray "* 2"
    ...
    db1 = 4. / m * np.sum(dZ1, axis=1, keepdims=True)   # BUG: "4." should be "1."
```

To answer the question posed in the assignment — *can you get gradient check to declare your derivative computation correct?* — yes: remove the stray `* 2` from `dW2` and change the `4.` back to `1.` in `db1`. Rerun the check and the difference collapses to roughly `1e-7`, and gradient checking now prints "works perfectly fine." That is what "proving backprop works" means in practice: not a mathematical proof, but strong numerical evidence that your gradient matches an independent estimate to many decimal places.

> **WHAT A PASSING CHECK ACTUALLY SHOWS**
>
> A passing gradient check shows that your backprop gradient agrees with a numerical estimate derived independently from the cost function. It does not prove your *cost* is the right cost for the problem, nor that your model will perform well — only that, given your cost, the gradients you feed to gradient descent are computed correctly. That is precisely the reassurance the CEO asked for: the learning engine is wired up right.

7. How You Actually Use Gradient Checking

Gradient checking is a debugging tool, not part of training, and using it well means knowing when to switch it on and off. The lecture notes give clear, practical rules.

> **PRACTICAL RULES FOR GRADIENT CHECKING**

> **Use it only to debug, never during training.** The numerical approximation re-runs forward propagation twice for *every single parameter*, which is enormously slow. Check your backprop once, confirm it passes, then turn the check off and let fast backprop do the actual learning.
>
> **If it fails, look at the components to find the bug.** When the difference is large, inspect which entries of `gradapprox` differ most from `grad`. If the mismatches all fall on the bias terms, suspect your `db` code; if on the weights of one layer, look there. This is exactly how you would hunt down the `dW2` and `db1` bugs above.
>
> **Remember the regularization term.** If your cost includes an L2 penalty, your gradient must include its derivative too — check the full, regularized gradient.
>
> **It does not work with dropout.** Dropout randomly changes the network each iteration, so there is no single fixed cost `J` to approximate. Turn dropout off, check the gradient, then turn dropout back on for training.

8. Putting It All Together

So, how do you use gradient checking and what are its components? The workflow is now clear. After writing forward and backward propagation, you assemble three pieces: the trusted **forward propagation** that gives the cost, the **backward propagation** you want to verify, and the **check** that approximates each gradient numerically from the cost alone and compares it to backprop's answer. You flatten all parameters and gradients into matching vectors, loop over every parameter taking a two-sided difference, and collapse the comparison into a single relative difference. If that number is below roughly `1e-7`, you trust your gradients; if not, you inspect the components, find the bug, and rerun.

> **THE MAIN TAKEAWAYS**
>
> Gradient checking verifies the closeness between the gradients from backpropagation and a numerical approximation of the gradient computed from forward propagation. It works because the two methods are independent, so a bug in one cannot hide in the other. It is slow, so you run it only to confirm your code is correct, then turn it off and use backprop for the actual learning. Used this way, it is one of the most reliable bug-finding tools in all of deep learning.

The deepest lesson is one every practitioner learns sooner or later: a network that appears to train is not the same as a network whose gradients are correct. Silent gradient bugs can quietly cap your model's performance with no error message to warn you. Gradient checking turns that invisible risk into a single, trustworthy number — and that is how you give your CEO, and yourself, real proof that backpropagation is working.